

COMP 356
Programming Language Structures
Notes for Chapter 4 of *Concepts of Programming Languages*
Scanning and Parsing

The **scanner** (or **lexical analyzer**) of a compiler processes the source program, recognizing tokens and returning them one at a time.

The **parser** (or **syntactic analyzer**) of a compiler organizes the stream of tokens (from the scanner) into parse trees according to the grammar for the source languages.

Separating the scanner and parser into different stages of the compiler:

- makes implementing the parser easier
- increases portability of the compiler
- permits parallel development
- eases tool support

1 Scanning

Scanning is essentially pattern matching - each token is described by some pattern for its lexemes.

Examples:

- the pattern for an identifier (token id) is a letter followed by a sequence of letters, digits and underscores. Lexemes include: i3, foo, foo_7, ...
- the pattern for a floating point numeric literal (token num) is an optional sign, followed by a sequence of digits, followed by a decimal point, followed by a sequence of digits. Lexemes include: -7.3, 3.44, ...

1.1 Regular Expressions

regular expressions are a notation for defining patterns.

<u>notation</u>	<u>name</u>	<u>meaning</u>	<u>example</u>	<u>matches</u>
[...]	character class	any of a set	[a-zA-Z]	any letter
*	Kleene * Kleene closure	0 or more occurrences	a*	0 or more a's
+	Kleene + positive closure	1 or more occurrences	a+	1 or more a's
?	optional	0 or 1 occurrences	a?	a or ϵ
	alternative		a b	a or b
(...)	grouping		a(b c)	a followed by b or c

Table 1: Regular expression notation

Because | has lowest precedence, **ab | c** (the last example above with the parentheses removed) matches **ab** or **c**, and not **a** followed by **b** or **c**.

A **regular definition** is a named regular expression. Such names can be used in place of the regular expression in following definitions.

Examples:

```
letter = [a-zA-Z]
digit = [0-9]
id = {letter}{letter} | {digit} | _)*
num = (+ | -){digit}+.{digit}+
```

The name of a regular definition is put in { } on the R.H.S. of another regular definition. This is to distinguish to use of the named definition, for example, {digit}, from the simple pattern given by the letters in the name, for example, d followed by i followed by g followed by i followed by t.

The **language** specified by a regular expression is the set of all strings that match the pattern given by the regular expression. For example, the language specified by the pattern for id given above includes strings such as foo, i3, foo_7 and so on.

A **regular language** is any language that can be specified by a regular expression. Every regular languages is context free (can be specified by a BNF definition), but not vice-versa. For example:

$$\{a^n b^n \mid n \geq 0\}$$

is a context free language, because it is defined by the following grammar:

$\langle S \rangle \rightarrow a \langle S \rangle b \mid \epsilon$

but is not a regular language.

Hence, regular expressions are strictly less powerful than context free grammars (BNF definitions). However, they can be used to generate more efficient pattern matchers.

To generate a pattern matcher from regular expressions, tools such as lex and JFlex first convert regular expressions to state transition diagrams, and then generate pattern matching code from the diagrams.

1.2 Using JFlex

JFlex is a tool for generating Java implementations of scanners from regular expression definitions of tokens.

To configure your account for using JFlex (and CUP), first download the file `cup.jar` from the course web page (see the links for Homework 3). Remember which directory this file is stored in. Next, use a text editor such as pico to create/edit a file called `.profile` in your home directory, add the following content to your `.profile`, and save it:

```
CLASSPATH=~ /cup.jar:.
export CLASSPATH
PATH=.: $PATH
export PATH
```

If you saved `cup.jar` somewhere other than your home directory, the first line should include the appropriate directory. For example, if you saved `cup.jar` to your `Desktop` directory, the first line should be:

```
CLASSPATH=~ /Desktop/cup.jar:.
```

Your `PATH` and `CLASSPATH` environment variables are now set correctly for all of the software we will use in this course.

A JFlex specification is put in a file with a `.lex` extension, i.e. `example.lex`. Assuming that `example.lex` contains a valid JFlex specification, the command:

```
java JFlex.Main example.lex
```

generates the scanner code and places it in file `Yylex.java`. (Any error messages about inability to find `main` or `JFlex.Main` indicate that your `CLASSPATH` environment variable is not set correctly.)

The scanner code is in class `Yylex`. The constructor for class `Yylex` takes a stream reader (use `System.in` for standard input) as the input stream to read the source program from.

The other significant method in class `Yylex` is `next_token()`, which returns the next token. For use with CUP (a parser generator), `next_token()` should return instances of class `Symbol` from `java_cup.runtime.Symbol` (which requires that CUP is installed and your `CLASSPATH` is set correctly).

Class `Symbol` has two significant fields (data members):

- `int sym` - the token recognized (represented as an integer)

- **Object value** - can contain instances of any Java class, and is often used to hold lexemes as instances of classes such as `String`, `Double` and `Integer`. CUP generates parsers from attribute grammars, and this `value` field will become an intrinsic attribute of the token in the attribute grammar.

The one-argument constructor of class `Symbol` takes the token (of type `int`) as its parameter, while the two-argument constructor for class `Symbol` has parameters for the values of both of these fields.

The token values should be defined as integer constants in class `sym`. CUP generates this class automatically, or it can be constructed by hand if CUP is not being used.

We will construct a JFlex scanner and a CUP parser for the following simple “calculator” language. This language defines lists of arithmetic expressions, and the output from our parser will be the value of each expression.

```
<exprl> → <exprl> <expr> ; | ε
<expr> → num | <expr> addop <expr> | <expr> mulop <expr> | (<expr>)
```

Format of a JFlex specification file:

```
user code
%%
JFlex directives
%%
regular expression rules and actions
```

For an example, see file `example.lex` on the course web page.

The contents of the *user code* section are copied directly into the generated scanner code. For example, imports of any needed Java classes would go here.

The *JFlex directives* section is used to customize the behavior of JFlex. Some common directives include:

```
%cup // to specify CUP compatibility
// the following three lines make JFlex recognize EOF properly
%eofval{
    return new Symbol(sym.EOF);
%eofval}
```

Notes on JFlex directives:

- the value of `sym.EOF` must be 0 (this only matters if you construct class `sym` by hand)
- every directive must start at the beginning of a line
- the line breaks in the macro defined with `%eofval` above are critical

Any regular definitions also go in this section. These use exactly the syntax of regular definitions presented in class, except that double quotes (" ") are used instead of single quotes (' ') to distinguish metasympols from literal characters.

The *regular expression rules and actions* give a sequence of regular expression patterns for tokens and actions to be executed when the associated actions are matched. Notes:

- each action is Java code, and Java keyword `return` can be used to specify the value returned by method `next_token()`
- white space (spaces, tabs, ...) in the source program is ignored by giving a pattern for white space with an empty action. Since the action contains no `return`, method `next_token()` ignores white space and keeps scanning the source program until something else is found.
- the patterns are tried in order, and the first matching pattern is used.

- a period (.) is a pattern that matches any character. This is useful for reporting lexical errors. However, this also means that the character period must be put in double quotes, i.e. ".", in a pattern if you mean the character . and not the metasymbol.
- in each action, the string of characters that matched the associated pattern (the lexeme) can be accessed using function `yytext()`

2 Bottom-Up Parsers

The **parser** (or **syntactic analyzer**) of a compiler organizes the stream of tokens (from the scanner) into parse trees according to the grammar for the source languages.

All grammars used in designing parsers must be unambiguous. Choices:

- remove ambiguity by hand (by modifying the grammar)
- use “disambiguating rules” to specify which choices to make when constructing the parse tree.

Parsing algorithms that can parse any unambiguous grammar are $O(n^3)$, where n is the length of the string being parsed. Commercial compilers use $O(n)$ (linear) parsing algorithms that can handle specific classes of grammars, called LL(1) and LR(1) grammars. These grammars are sufficient to describe the syntax of the vast majority of programming language features.

Parsers are classified as:

- **top-down** parsers, which build parse trees from the root to the leaves. These parsers are designed using LL(1) grammars.
- **bottom-up** parsers, which build parse trees from the leaves to the root. These parsers are designed using LR(1) grammars.

Bottom-up parsers build parse trees from the leaves to the root. The idea is to use the grammar rules “backward” - the parser matches against the R.H.S. of grammar rules and when some rule is matched, it is used to construct a subtree with the R.H.S. of the rule as the children and the L.H.S. as the root.

A bottom-up parser uses a stack to keep track of the parse. At each parsing step, there are two possible actions:

- **shift** - push the next token from the input onto the stack
- **reduce** - if the sequence of grammar symbols on “top” of the stack matches the R.H.S. of a grammar rule, pop them all and replace them with the L.H.S. of that rule.

The parser is controlled by a table generated from the grammar. The tables are too large and complex to construct by hand, and so are generated using a tool such as yacc or CUP.

Bottom-up parsers are also called LR parsers, because they parse LR grammars.

L - Left to right scan of the input

R - generates a Rightmost derivation (in reverse)

Every LL(1) grammar is an LR grammar, but not vice versa:

LL(1) grammars $\subset$ LR grammars

For example, many left recursive grammars are LR. Hence, bottom-up parsers have more parsing power than top-down parsers.

The parsers generated by yacc and CUP are LALR(1) parsers, where LALR(1) stand for Look-Ahead LR(1). These parsers are less powerful than LR parsers, i.e.:

LL(1) grammars $\subset$ LALR(1) grammars $\subset$ LR grammars

but the loss of parsing power is small and the parse tables for LALR(1) parsers are much smaller than those for LR parsers.

If the grammar used as input to yacc or CUP is ambiguous or otherwise not LALR(1), these tools will report **parsing conflicts** as error messages. The possible conflicts are:

- **shift-reduce conflicts** - at some step, the parser can't tell whether to shift a token onto the stack or do a reduction.
- **reduce-reduce conflicts** - at some step, the parser can't tell which of two reductions to perform. This can occur when two rules in the grammar have identical R.H.S.'s, but different L.H.S.'s.

2.1 Using CUP

CUP generates an LALR(1) parser (in Java) from an attribute grammar specification:

- this approach works well with synthesized attributes
- each time a rule is used in a reduction, the attribute of the L.H.S. of the rule is computed (via user-specified Java code) from the attributes of the R.H.S.
- bottom-up parsing ensures that the attributes of the R.H.S. are available when needed.

JFlex scanners can easily be made compatible with CUP parsers.

A CUP specification is placed in a file with extension `.cup`, for example, `example.cup`. To generate parser code:

```
java java_cup.Main example.cup
```

This generates:

- `parser.java` which contains the parser code
- `sym.java` which contains definitions of all of the tokens used as integer constants. These must match the tokens used in the scanner.

To compile and run the parser:

1. ensure that the scanner code generated using JFlex (for example, `Yylex.java`) is in the same directory as `parser.java` and `sym.java`
2. compile by executing:

```
javac parser.java
```

3. run the parser by executing:

```
java parser < input
```

where `input` is the file containing the source program.

Or, to enter the source program directly from the keyboard, just enter:

```
java parser
```

and then type the source program and press Ctrl-D when finished.

Format of a CUP specification file:

```
imports
directives
declarations of tokens and nonterminals
grammar rules and actions
```

For an example, see file `example.cup` on the course web page or the class handout.

The *imports* are copied directly to the beginning of the generated code in `parser.java`. For example,

```
import java_cup.runtime.*;
```

is usually placed here to provide access to the needed library classes.

The *directives* supply user code to be placed in the indicated class in the generated code. For example:

```
parser code { : code : }
```

causes the code in the special brackets { : : } to be placed in class **parser**.

In the *declarations of tokens and nonterminals* section, all grammar symbols are declared as follows:

- the name used for a token must match the name assigned to the **sym** field of class **Symbol** when that token is recognized by the scanner.
- if a token has an attribute (placed in the **value** field of class **Symbol** by the scanner), the type of that token must be declared and must match the attribute value assigned by the scanner.
- if a nonterminal has an attribute, the type of that attribute must also be given. The majority of nonterminals will have attributes.
- by convention, token names are in all caps and nonterminal names are all lowercase. Note that non-terminal names are not put in angle brackets (< >).

This section also includes any specifications of associativity and precedence of tokens. These specifications are actually disambiguating rules and permit parsing of many ambiguous grammars when used appropriately. The syntax of these specifications is:

```
precedence <spec> terminal { , terminal };
```

where:

```
<spec> → left | right | nonassoc
```

The <spec> specifies associativity. If a token is declared as **nonassoc**, then it can not occur multiple times in one expression. For example, $1 < 2 < 3$ is illegal in many programming languages.

Precedence rules:

- all tokens listed in the same declaration have the same precedence.
- if there are multiple precedence declarations, later declarations have higher precedence.
- all tokens not listed in a **precedence** declaration have lowest precedence

For example:

```
precedence left ADDOP
precedence left MULOP
```

declares that **MULOP** has higher precedence than **ADDOP**, and that both are left associative.

In many cases, these precedence and associativity declarations can be used to remove shift-reduce and reduce-reduce conflicts from the grammar.

The *grammar rules and actions* section is an attribute grammar for the language being parsed:

- the L.H.S. of the first rule is the start symbol
- the symbol **:=** is used in place of $\rightarrow$
- nonterminal names are not in < > (the declarations specify which names are nonterminals and which are tokens)
- each rule ends in ;

- each symbol on the R.H.S. of a rule can be labeled using `:` and a name, i.e. `expr:l ADDOP:op expr:r`, where `l`, `op` and `r` are labels
- for tokens, the label is used to refer to the intrinsic attribute assigned by the scanner (stored in the `value` field of the `Symbol`)
- for nonterminals, the attribute value is computed using **actions** associated with each rule
- the attribute of the L.H.S. of the rule is always `RESULT`

The actions associated with each rule:

- are in special brackets `{: :}`
- contain Java code that can refer to attribute labels and `RESULT`
- typically compute the value of `RESULT` based on the attribute values of the grammar symbols on the R.H.S. of the rule. Hence, the attributes are synthesized attributes.
- are executed each time the rule is used in a reduction
- can easily be used to build parse trees or do other computation

The commands needed to compile and run a CUP parser with a JFlex scanner are as follows, assuming:

- file `example.lex` contains the JFlex specification
- file `example.cup` contains the CUP specification
- file `input` contains the source program
- JFlex and CUP are installed and the `CLASSPATH` is set correctly

<u>command</u>	<u>comment</u>
<code>java JFlex.Main example.lex</code>	creates <code>Yylex.java</code>
<code>java java_cup.Main example.cup</code>	creates <code>sym.java</code> and <code>parser.java</code>
<code>javac parser.java</code>	compiles everything
<code>java parser < input</code>	runs the parser

Table 2: Commands needed to compile and run a CUP parser with a JFlex scanner.

To add tokens, you must declare them in the `.cup` file and also add actions to the `.lex` file to recognize and return them. Also, you must run CUP (to generate constant declarations for the new tokens in `sym.java`) before recompiling anything.